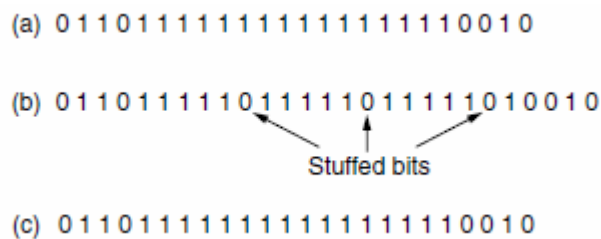


Experiment No 5

Aim: Write a program to implement Bit-Stuffing in a frame.

1. Flag bits with bit stuffing

The third method of delimiting the bit stream gets around a disadvantage of byte stuffing, which is that it is tied to the use of 8-bit bytes. Framing can be also at the bit level, so frames can contain an arbitrary number of bits made up of units of any size. Each frame begins and ends with a special bit pattern, 01111110 or 0x7E in hexadecimal. This pattern is a flag byte. Whenever the sender's data link layer encounters five consecutive 1s in the data, it automatically stuffs a 0 bit into the outgoing bit stream. This **bit stuffing** is analogous to byte stuffing, in which an escape byte is stuffed into the outgoing character stream before a flag byte in the data. When the receiver sees five consecutive incoming 1 bits, followed by a 0 bit, it automatically de-stuffs (i.e., deletes) the 0 bit. If the user data contain the flag pattern, 01111110, this flag is transmitted as 011111010 but stored in the receiver's memory as 01111110. Figure gives an example of bit stuffing. With bit stuffing, the boundary between two frames can be unambiguously recognized by the flag pattern. Thus, if the receiver loses track of where it is, all it has to do is scan the input for flag sequences, since they can only occur at frame boundaries and never within the data.



```

def bit_stuffing(data):
    stuffed_data = ""
    count = 0
    for bit in data:
        stuffed_data += bit
        if bit == '1':
            count += 1
        else:
            count = 0
        # Insert a 0 after five consecutive 1s
        if count == 5:
            stuffed_data += '0'
            count = 0
    return stuffed_data

# Main program
print("BIT STUFFING")
data = input("Enter the data bits (0s and 1s): ")
# Check whether input contains only 0 and 1
if all(bit in '01' for bit in data):
    stuffed_data = bit_stuffing(data)
    print("\nOriginal Frame : ", data)
    print("Stuffed Frame : ", stuffed_data)
else:
    print("Invalid input! Please enter only 0 and 1.")

```

'0' in '01' $\rightarrow$ True

'1' in '01' $\rightarrow$ True

'2' in '01' $\rightarrow$ False

'a' in '01' $\rightarrow$ False